

Analysis of Different Starting Balatro Strategies with SimPy

By: Zachary Allen (Project Group 391)

July 21st, 2025

Abstract

Balatro is an extremely in-depth roguelike deck builder game that combines poker hands with scoring multipliers. Players progress through increasingly difficult blinds using strategic hand selection and resource management. This report presents a simulation-based analysis of eight distinct starting strategies under a simplified ruleset that excludes jokers, planets, spectrals, and boss encounters to isolate core strategic decision-making patterns. The simulation framework implements three difficulty levels – easy, medium, and hard – based on different playable modes from the game. Each of the modes are characterized by varying constraints on hands per blind, discards per blind, and escalating blind targets across three antes. This structure enables a systemic comparison of strategy performance under different resource constraints and pressure scenarios. Across 500 simulated games for each strategy-difficulty combination the Flush Chaser strategy, which prioritizes building flush hands over any other type, results in the highest average total score before failure and has a statistically significant advantage over the next best strategies at each difficulty at $p < 0.05$.

Background and Description of the Problem

Balatro, a 2024 nominee for Video Game of the Year and winner of Best Debut Indie Game, is a poker-inspired roguelike deckbuilder which involves playing poker hands to score points, but with a twist that you can modify your deck and use special Joker cards and other purchasable modifiers that provide powerful synergies and multipliers to progress through increasingly difficult score targets (called Blinds) [1].

Developing a consistent strategy for Balatro is difficult if not impossible since the Jokers and other modifiers you can purchase after completing a Blind are randomized, making each run of the game unique. Deciding to avoid these items will prevent one's ability to scale and result in an early game over.

This project aims to simulate how far different starting strategies can progress in a simplified Balatro environment (no special Jokers, modifiers, shops, bosses, etc.) under varying game configurations before failure. Since early game decisions often determine whether you survive long enough to find powerful synergies in the shop, knowing which base strategies are most reliable would help players make better choices when they haven't found their "build" yet or when their current strategy gets disrupted.

This report will review past explorations into Balatro simulations, describe how the code is structured, show how to run the program, analyze the results of the simulation, and conclude with overall findings and future ideas for this simulation.

Main Findings

Past Efforts

While this specific evaluation of starting Balatro strategies has not been done before, related simulations have been conducted before such as:

- Simulating the probability of drawing a certain hand type for different starting decks [2]
- Simulating the odds of drawing a certain hand type (with starter deck) while factoring discards and specialty jokers [3]

The second simulation is particularly interesting as it provides the number of discards required until one can be at least 90% confident that a specific hand type will be drawn. For example, it takes 3 discards on a standard deck to be 90% confident a flush will be drawn, whereas it takes 5 discards for three of a kind, straight, and full house. These findings will be relevant when analyzing the results of this simulation.

Code Structure

Game Rules

The code is structured to follow the basic game rules:

- Start with a standard deck of 52 cards
- Each round (called an "**Ante**"), you will face 3 "**Blinds**" (Small, Big, and Boss)
- Each Blind has a target score you must reach. If you fail to reach the target score you lose and the game is over.

- Each Blind you have a limited number of **hands** (pair, two-pair, straight, flush, etc.) to play. You can play up to 5 cards per hand and any cards you play will be removed from the deck of 52. If you don't like your current set of cards, you also have a limited number of **discards** per blind where you can swap up to 5 cards at a time for random cards from your current deck.
- When you play a hand, your score is calculated as $(\text{Base Chips} \times \text{Multiplier})$. Different poker hands have different base chips and multipliers. The cards you play contribute their face value in chips (face cards = 10, Ace = 11)
 - Example: Two Pair has a base scoring of $(20 \text{ Chips} \times 2 \text{ Multiplier})$. If you play a two-pair of Aces and Eights your score would be $(20 + 11 + 11 + 8 + 8) \times 2 = 116$.

Strategy Definitions

The eight strategies being tested and how each of them are designed are provided in the table below:

Strategy	Description
High Card Focus	Always play the five highest cards based on rank.
Pair Hunter	Check if current hand has pair or better, if so play it plus best remaining cards. If not, discard cards that don't contribute to pairs.
Conservative	Only play if five highest cards has a pair or better. Otherwise discard 3 lowest cards.
Aggressive	Almost always play five highest cards unless hand is poor and plenty of discards remain.
Flush Chaser	Always aim to build flushes. Discard decisions are dependent on the suits in current hand and in the remaining deck.
Straight Chaser	Always aim to build straights. Discard decisions are dependent on which straight is most likely in current hand and the ranks in the remaining deck.
High Value	If there are 3+ high cards (10, J, Q, K, A) in hand, discard until at least 5 high cards are in hand then play. Otherwise, play five highest cards available.
Balanced	If five best cards are strong (straight, flush, full house, etc.) play it. If five best cards are good (three of a kind, two pair) play unless it is a weak three of a kind and 2+ discards and hands remain. If five best cards only have a pair, play unless the pair is low and 2+ discards and hands remain.

Difficulty Definitions

The three difficulties are defined using the table below:

Difficulty	Hands Per Blind	Discards Per Blind	Blind Levels (3 per Ante)
Easy	5	4	Ante 1: 300, 450, 600 Ante 2: 800, 1200, 1600 Ante 3: 2000, 3000, 4000
Normal	4	5	Ante 1: 300, 450, 600 Ante 2: 900, 1350, 1800 Ante 3: 2600, 3900, 5200
Hard	4	4	Ante 1: 300, 450, 600 Ante 2: 1000, 1500, 2000 Ante 3: 3200, 4800, 6400

The blind levels are based on the White (Easy), Green (Medium), and Purple (Hard) stakes [4] from the game. The hands/discards per blind are based on the Blue (Easy) and Red (Medium) decks [5]. All other decks use the Hard configuration (4 hand/discards per blind).

How A Game Simulation is Run

For a given strategy-difficulty combination, a game simulation occurs by drawing 8 cards from a standard shuffled deck of 52 cards. The strategy's `decide_action` method is called, which will ultimately decide whether to `play` or `discard` and which cards to do so with. If a hand is played, its score is calculated. If the total score for the blind exceeds the target, move on to the next blind and reset the deck. If no playable hands remain and the target score for the blind is not exceeded, the game is ended, and the overall score across all blinds is recorded for further analysis.

How to Run the Simulation Code

Install the `pandas`, `numpy`, `simpy`, and `openpyxl` Python packages into your environment. In the last line of `balatro_sim.py`, set the number of simulations you wish to run for each strategy-difficulty combination (default = 500). Then run the `balatro_sim.py` file. Once it is complete, an Excel file will be created with every action played for each strategy. The terminal will print out the summary statistics for each strategy with 95% confidence intervals for average total score.

Results

Across 500 simulations per strategy-difficulty combination, the Flush Chaser strategy performed significantly better than all other strategies across all difficulties. The average total scores across all simulations can be shown in the confidence plot below.

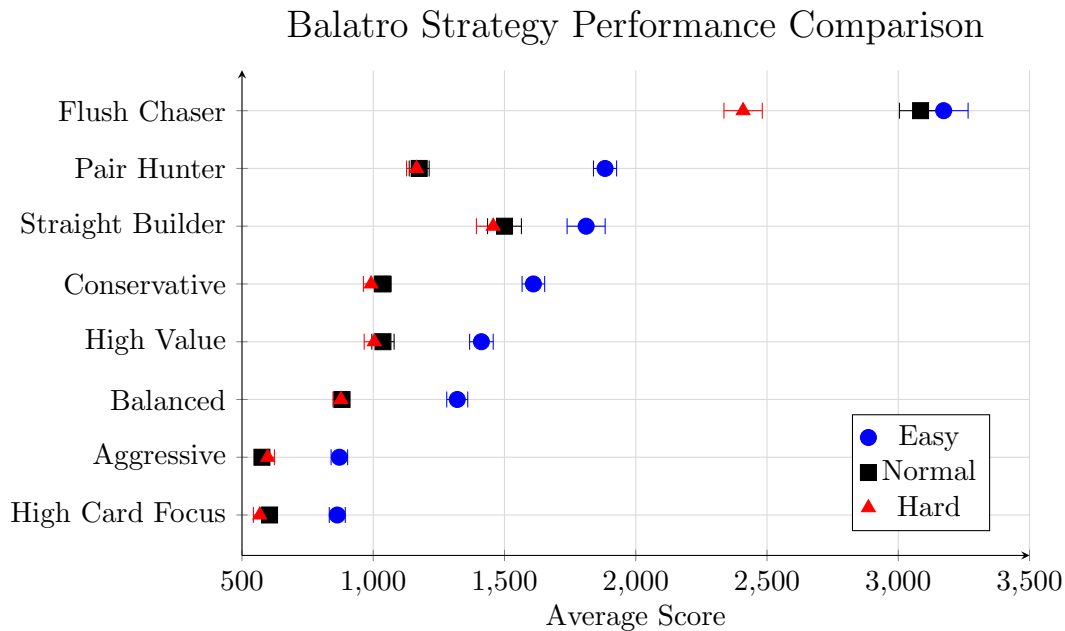


Figure 1: Balatro Strategy Performance Comparison across Easy, Medium, and Hard Difficulties. Error bars represent 95% Confidence Interval for Average Total Score.

All pair-wise differences in average scores between strategies were also computed. For brevity, the following table shows all cases where the difference in two strategies **were not** statistically different from each other ($p \geq 0.05$).

Strategy 1	Strategy 2	Score Diff	t-stat	p-value
Pair Hunter(E)	Straight Builder(E)	+72	+1.67	0.0952
High Card Focus(E)	Aggressive(E)	-9	-0.39	0.6939
High Card Focus(M)	Aggressive(M)	+30	+1.69	0.0914
Conservative(M)	High Value(M)	-1	-0.03	0.9762
High Card Focus(H)	Aggressive(H)	-30	-1.66	0.0973
Conservative(H)	High Value(H)	-13	-0.51	0.6127

Conclusion

In summary, by completing this project I learned a lot about how to write and structure code for the purposes of simulation. By switching to an object-oriented approach after Milestone 2, it became very easy to gather, calculate, and analyze output. On a more technical level, the Flush Chaser strategy being as good as it is aligns with simulations others have done previously. Specifically, the second one mentioned in the Past Efforts section which factored discards, showed that it takes less discards to get flushes (3) than three of a kind, straight, and full house (5) at 90% certainty. Since flushes are worth more than straights in Balatro, it makes sense that on average it would perform better given the strategy implementation. Another interesting finding is that Pair Hunter and Straight Builder are not significantly different on Easy but are on Medium/Hard.

Future work could improve upon my simulation by incorporating an income and shop system. This would require a massive coding effort, since there are hundreds of Jokers and other special cards with different conditional logic and rules, but would allow this simulation to scale well past 3 antes worth of blinds. It could also be possible to test one joker at a time to measure its relative strength.

References

- [1] Balatro Wiki. (2025). Balatrowiki.org. <https://balatrowiki.org/>
- [2] R/Balatro on reddit: I've made a python simulation! I present to you, the impact of hand size and starting deck! (see comments). (mangoboss42). https://www.reddit.com/r/balatro/comments/1b40a3d/ive_made_a_python_simulation_i_present_to_you_the/
- [3] Apocalypse Hero. (2024, Mar 15). How effective are discards? | Balatro [Video]. YouTube. <https://www.youtube.com/watch?v=WQdLDMLrYIA>
- [4] Stakes - Balatro Wiki. (2025). Balatro Wiki. <https://balatrowiki.org/w/Stakes>
- [5] Decks - Balatro Wiki. (2025). Balatro Wiki. <https://balatrowiki.org/w/Decks>

LLM Usage

Large Language Models were used in this project only for: creating the Figure 1 Tikz plot seen in the report faster, adding additional code comments for readability, and brainstorming ideas for the Balanced strategy (the final version was fairly different from what it provided initially).